

TRAINING INSTITUTE MANAGEMENT SYSTEM

Mini Project – Database Management System

B.Sc. Data Science

Semester III

SDBI

Submitted By,

Sanskriti Wadekar

Roll No.: 21003

Academic Year: 2026–2027

Introduction

The Training Institute Management System is a database management project designed to manage the activities of a training institute in an organized manner. The system maintains information about students, trainers, courses, batches, enrollments, attendance and assessment results.

The database helps the institute store and retrieve information efficiently and reduces the problems associated with maintaining records manually. It also provides useful reports related to student enrollment, attendance, assessment performance and training progress.

MySQL/MariaDB is used to implement the database, while SQL queries are used to retrieve and analyse the stored information.

Objective

The main objective of the Training Institute Management System is to design and implement a relational database for managing the activities of a training institute.

The objectives of the project are:

1. To maintain student information.
2. To maintain trainer information.
3. To store course details and fees.
4. To manage batches and their schedules.

5. To record student enrollments.
6. To maintain student attendance records.
7. To store assessment results.
8. To generate useful reports using SQL queries.
9. To analyse student performance and attendance.
10. To provide a student-wise training progress report.

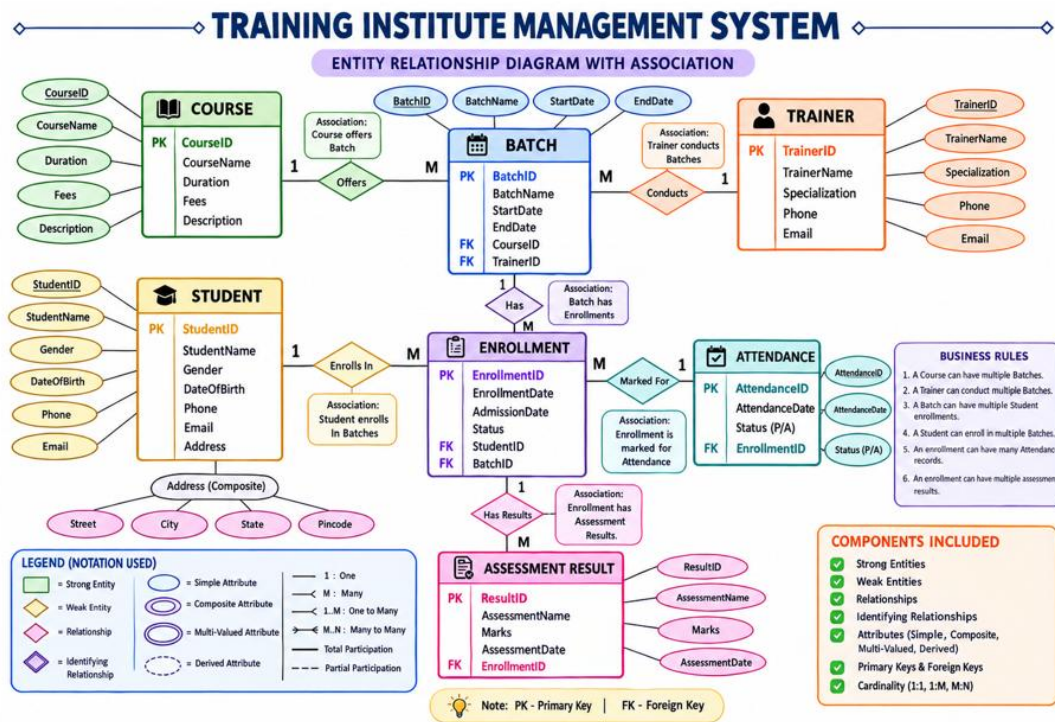
Database Design

The database is designed using a relational database model. The system contains seven main tables:

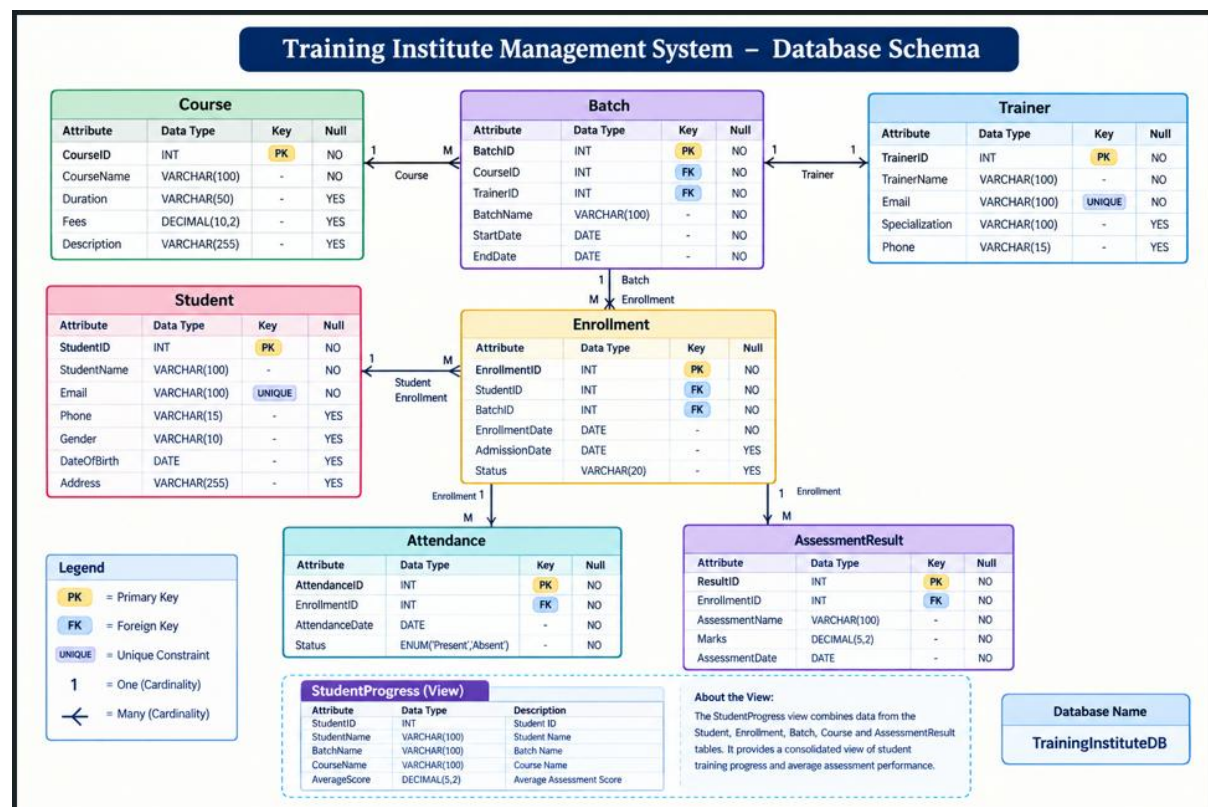
1. Student
2. Trainer
3. Course
4. Batch
5. Enrollment
6. Attendance
7. AssessmentResult

A view named **StudentProgress** is also created to generate a student-wise training progress report.

Entity Relationship Diagram of Training Institute Management System:



Database Schema Diagram



Student Table:

Field	Description
StudentID	Primary Key
StudentName	Name of student
Email	Student email
Phone	Contact number
Gender	Gender
DateOfBirth	Date of birth
Address	Student address

Trainer Table

Field	Description
TrainerID	Primary Key
TrainerName	Trainer name

Email	Trainer email
Specialization	Area of specialization
Phone	Contact number

Course Table

Field	Description
CourseID	Primary Key
CourseName	Name of course
Duration	Course duration
Fees	Course fees
Description	Course description

Batch Table

Field	Description
BatchID	Primary Key
CourseID	Foreign Key
TrainerID	Foreign Key
BatchName	Batch name
StartDate	Starting date
EndDate	Ending date

Enrollment Table

Field	Description
EnrollmentID	Primary Key
StudentID	Foreign Key
BatchID	Foreign Key
EnrollmentDate	Enrollment date
AdmissionDate	Admission date
Status	Enrollment status

Attendance Table

Field	Description
AttendanceID	Primary Key
EnrollmentID	Foreign Key
AttendanceDate	Attendance date
Status	Present/Absent

AssessmentResult Table

Field	Description
ResultID	Primary Key
EnrollmentID	Foreign Key
AssessmentName	Assessment name
Marks	Assessment marks
AssessmentDate	Assessment date

Relationships Between Tables:

Relationship	Type
Course → Batch	One-to-Many
Trainer → Batch	One-to-Many
Student → Enrollment	One-to-Many
Batch → Enrollment	One-to-Many
Enrollment → Attendance	One-to-Many
Enrollment → AssessmentResult	One-to-Many

Primary keys uniquely identify records in each table, while foreign keys establish relationships between related tables and maintain referential integrity.

Student Sample Data:

StudentID	StudentName	Gender	Email
1	Rahul Sharma	Male	rahul@gmail.com
2	Priya Patil	Female	priya@gmail.com
3	Amit Joshi	Male	amit@gmail.com
4	Sneha Kulkarni	Female	sneha@gmail.com
5	Rohan Deshmukh	Male	rohan@gmail.com

The database contains 10 student records, 4 trainer records, 4 course records, 6 batch records, 15 enrollment records, 49 attendance records and 30 assessment result records.

Display Students with Their Batch and Course:

Purpose: Displays student enrollment details along with batch and course information.

QUERY

SELECT

b.BatchName,

s.StudentID,

s.StudentName,

c.CourseName

FROM Batch b

JOIN Enrollment e

ON b.BatchID = e.BatchID

JOIN Student s

ON e.StudentID = s.StudentID

JOIN Course c

ON b.CourseID = c.CourseID

ORDER BY b.BatchID, s.StudentID;

Server: 127.0.0.1 » Database: traininginstitutedb » View: StudentProgress

Browse Structure SQL Search Export Privileges Operations

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all Number of rows: 25 Filter rows: Search this table

Extra options

BatchName	StudentID	StudentName	CourseName
PDS Morning Batch	1	Rahul Sharma	Python Data Science
PDS Morning Batch	2	Priya Patil	Python Data Science
PDS Morning Batch	3	Amit Joshi	Python Data Science
PDS Morning Batch	4	Sneha Kulkarni	Python Data Science
PDS Evening Batch	5	Rohan Deshmukh	Python Data Science
PDS Evening Batch	6	Anjali More	Python Data Science
PDS Evening Batch	7	Kunal Shah	Python Data Science
Web Development Batch	8	Neha Jadhav	Web Development
Web Development Batch	9	Aditya Pawar	Web Development
Web Development Batch	10	Pooja Gupta	Web Development
Cybersecurity Batch	1	Rahul Sharma	Cybersecurity
Cybersecurity Batch	5	Rohan Deshmukh	Cybersecurity
Cybersecurity Batch	8	Neha Jadhav	Cybersecurity
DBMS Batch	2	Priya Patil	Database Management
DBMS Batch	4	Sneha Kulkarni	Database Management

Display Batch, Course and Trainer Details

Purpose: Shows which trainer is assigned to each batch and the course being conducted.

QUERY

SELECT

b.BatchID,

b.BatchName,

c.CourseName,

t.TrainerName,

t.Specialization

FROM Batch b

JOIN Course c

ON b.CourseID = c.CourseID

JOIN Trainer t

ON b.TrainerID = t.TrainerID

ORDER BY b.BatchID;

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The left sidebar lists various databases and tables. The main panel displays a SQL query and its results.

SQL Query:

```
SELECT b.BatchID, b.BatchName, c.CourseName, t.TrainerName, t.Specialization FROM Batch b JOIN Course
Trainer t ON b.TrainerID = t.TrainerID ORDER BY b.BatchID;
```

Results:

BatchID	BatchName	CourseName	TrainerName	Specialization
1	PDS Morning Batch	Python Data Science	Amit Patil	Python and Data Science
2	PDS Evening Batch	Python Data Science	Amit Patil	Python and Data Science
3	Web Development Batch	Web Development	Snehal Joshi	Web Development
4	Web Development Weekend	Web Development	Snehal Joshi	Web Development
5	Cybersecurity Batch	Cybersecurity	Rahul Kulkarni	Cybersecurity
6	DBMS Batch	Database Management	Meena Deshmukh	Database Management

Total Students in Each Course

Purpose: Calculates the number of enrolled students for every course.

QUERY

```
SELECT
    c.CourseID,
    c.CourseName,
    COUNT(e.StudentID) AS TotalStudents
FROM Course c
JOIN Batch b
    ON c.CourseID = b.CourseID
JOIN Enrollment e
    ON b.BatchID = e.BatchID
GROUP BY
    c.CourseID,
    c.CourseName
ORDER BY c.CourseID;
```

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Course' table is selected. A SQL query is entered and executed, showing 4 rows. The query is:

```
SELECT c.CourseID, c.CourseName, COUNT(e.StudentID) AS TotalStudents FROM Course c JOIN Batch b ON c.CourseID = b.CourseID JOIN Enrollment e ON b.BatchID = e.BatchID GROUP BY c.CourseID, c.CourseName ORDER BY c.CourseID;
```

The results table shows the following data:

CourseID	CourseName	TotalStudents
1	Python Data Science	7
2	Web Development	3
3	Cybersecurity	3
4	Database Management	2

Batches Having More Than 2 Students

Purpose: Identifies batches with a relatively large number of enrolled students.

QUERY

SELECT

b.BatchID,

b.BatchName,

COUNT(e.StudentID) AS TotalStudents

FROM Batch b

JOIN Enrollment e

ON b.BatchID = e.BatchID

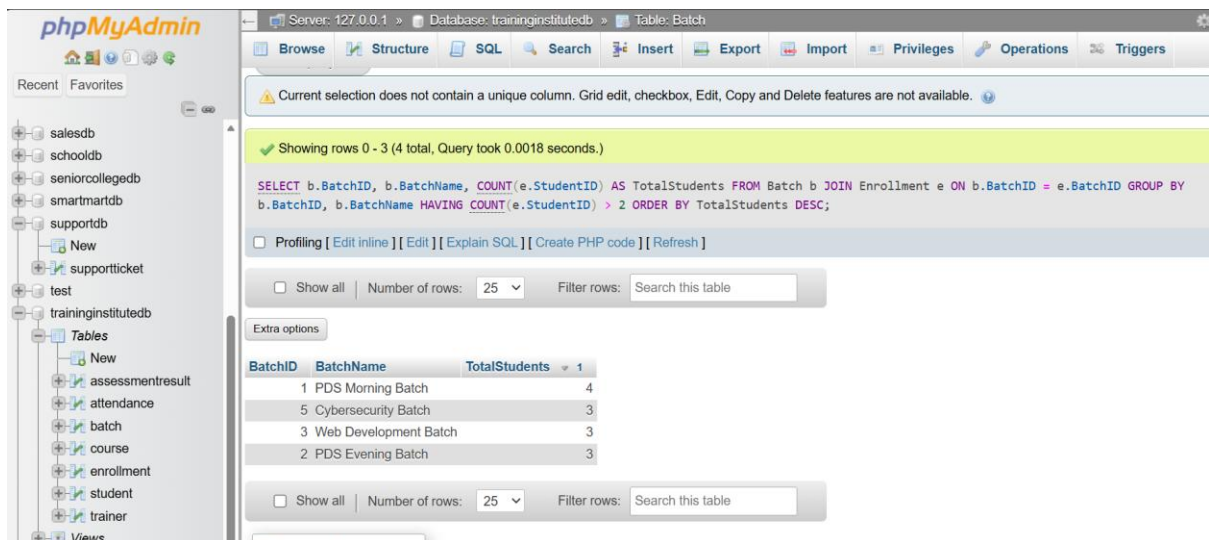
GROUP BY

b.BatchID,

b.BatchName

HAVING COUNT(e.StudentID) > 2

ORDER BY TotalStudents DESC;



Student Attendance Percentage

Purpose: Calculates the attendance percentage of each student.

QUERY

```
SELECT
    s.StudentID,
    s.StudentName,
    ROUND(
        (
            SUM(
                CASE
                    WHEN a.Status = 'Present' THEN 1
                    ELSE 0
                END
            ) * 100.0
        ) / COUNT(a.AttendanceID),
    2
```

) AS AttendancePercentage

FROM Student s

JOIN Enrollment e

ON s.StudentID = e.StudentID

JOIN Attendance a

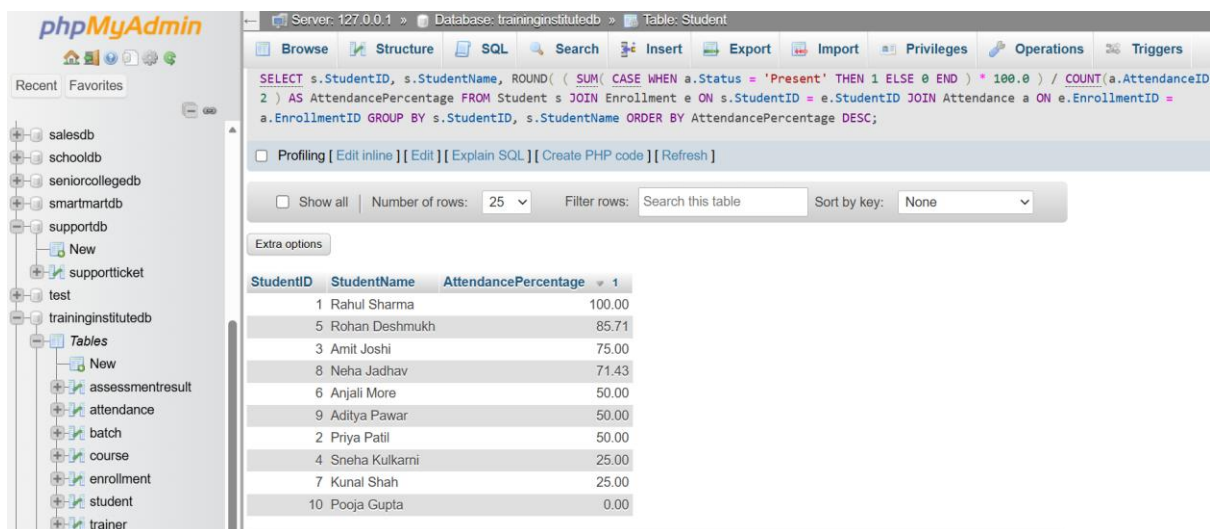
ON e.EnrollmentID = a.EnrollmentID

GROUP BY

s.StudentID,

s.StudentName

ORDER BY AttendancePercentage DESC;



The screenshot shows the phpMyAdmin interface with the following components:

- Left Panel:** A tree view of databases including salesdb, schooldb, seniorcollegedb, smartmartdb, supportdb, supportticket, test, and traininginstitutedb. The 'traininginstitutedb' database is selected, showing a list of tables: assessmentresult, attendance, batch, course, enrollment, student, and trainer.
- Top Panel:** Navigation tabs for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, and Triggers. The 'SQL' tab is active.
- SQL Query:** A query is entered in the text area:

```
SELECT s.StudentID, s.StudentName, ROUND( ( SUM( CASE WHEN a.Status = 'Present' THEN 1 ELSE 0 END ) * 100.0 ) / COUNT(a.AttendanceID 2 ) AS AttendancePercentage FROM Student s JOIN Enrollment e ON s.StudentID = e.StudentID JOIN Attendance a ON e.EnrollmentID = a.EnrollmentID GROUP BY s.StudentID, s.StudentName ORDER BY AttendancePercentage DESC;
```
- Results:** Below the query, there are options for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (a search box), and 'Sort by key' (set to None). Below these is an 'Extra options' button. The results are displayed in a table with the following data:

StudentID	StudentName	AttendancePercentage
1	Rahul Sharma	100.00
5	Rohan Deshmukh	85.71
3	Amit Joshi	75.00
8	Neha Jadhav	71.43
6	Anjali More	50.00
9	Aditya Pawar	50.00
2	Priya Patil	50.00
4	Sneha Kulkarni	25.00
7	Kunal Shah	25.00
10	Pooja Gupta	0.00

Average Assessment Score by Batch

QUERY

SELECT

b.BatchID,

b.BatchName,

ROUND(AVG(ar.Marks), 2) AS AverageScore

FROM Batch b

JOIN Enrollment e

ON b.BatchID = e.BatchID

JOIN AssessmentResult ar

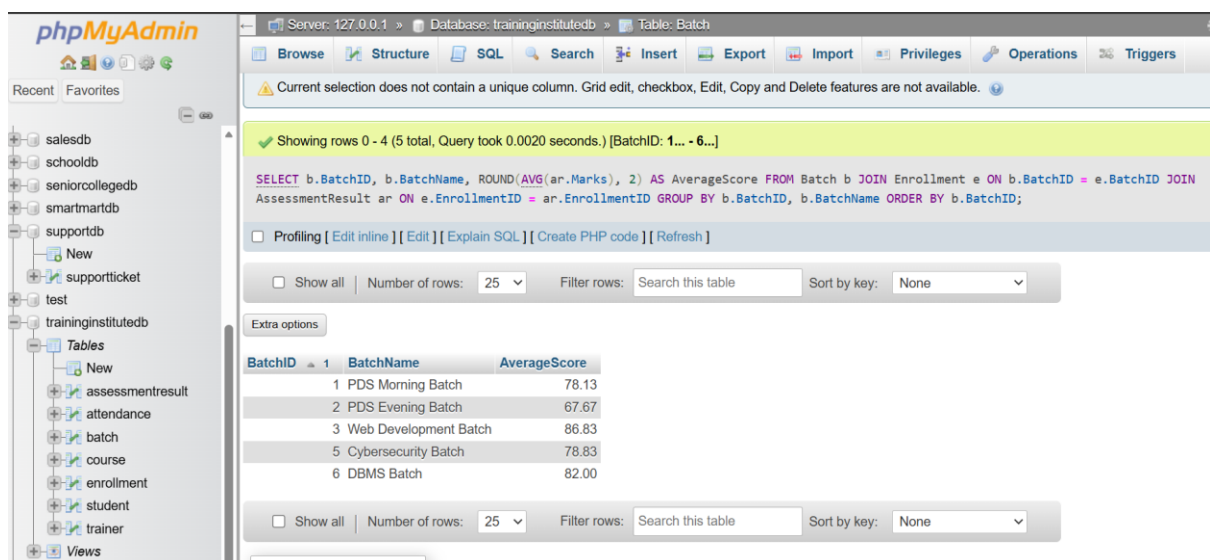
ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

b.BatchID,

b.BatchName

ORDER BY b.BatchID;



The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Batch' table is selected. The SQL query displayed is: `SELECT b.BatchID, b.BatchName, ROUND(AVG(ar.Marks), 2) AS AverageScore FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID JOIN AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID GROUP BY b.BatchID, b.BatchName ORDER BY b.BatchID;` The results show 6 rows of data for different batches, ordered by BatchID.

BatchID	BatchName	AverageScore
1	PDS Morning Batch	78.13
2	PDS Evening Batch	67.67
3	Web Development Batch	86.83
5	Cybersecurity Batch	78.83
6	DBMS Batch	82.00

Highest Performing Student in Each Batch

QUERY

WITH StudentScores AS (

SELECT

b.BatchID,

b.BatchName,

```

s.StudentID,

s.StudentName,

ROUND(AVG(ar.Marks), 2) AS AverageScore,

RANK() OVER (

    PARTITION BY b.BatchID

    ORDER BY AVG(ar.Marks) DESC

) AS StudentRank

FROM Batch b

JOIN Enrollment e

    ON b.BatchID = e.BatchID

JOIN Student s

    ON e.StudentID = s.StudentID

JOIN AssessmentResult ar

    ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

    b.BatchID,

    b.BatchName,

    s.StudentID,

    s.StudentName

)

SELECT

    BatchID,

    BatchName,

    StudentID,

    StudentName,

```

AverageScore

FROM StudentScores

WHERE StudentRank = 1

ORDER BY BatchID;

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Table: Batch' is selected. The SQL query is displayed in the main area, and the results are shown in a table below. The query is a complex one, using a CTE to calculate average scores and ranks for each batch, then selecting the top student (rank 1) from each batch.

Showing rows 0 - 4 (5 total, Query took 0.0026 seconds.)

```
WITH StudentScores AS ( SELECT b.BatchID, b.BatchName, s.StudentID, s.StudentName, ROUND(AVG(ar.Marks), 2) AS AverageScore, RANK() OVER ( PARTITION BY b.BatchID ORDER BY AVG(ar.Marks) DESC ) AS StudentRank FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID JOIN Student s ON e.StudentID = s.StudentID JOIN AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID GROUP BY b.BatchID, b.BatchName, s.StudentID, s.StudentName ) SELECT BatchID, BatchName, StudentID, StudentName, AverageScore FROM StudentScores WHERE StudentRank = 1 ORDER BY BatchID;
```

Extra options

	BatchID	BatchName	StudentID	StudentName	AverageScore
☐	1	PDS Morning Batch		Rahul Sharma	90.00
☐	2	PDS Evening Batch			83.50
☐	3	Web Development Batch	10	Pooja Gupta	93.00
☐	5	Cybersecurity Batch	1	Rahul Sharma	94.00
☐	6	DBMS Batch	2	Priya Patil	90.00

With selected: Edit Copy Delete

Students Who Have Not Attended Any Session

QUERY

SELECT

s.StudentID,

s.StudentName

FROM Student s

LEFT JOIN Enrollment e

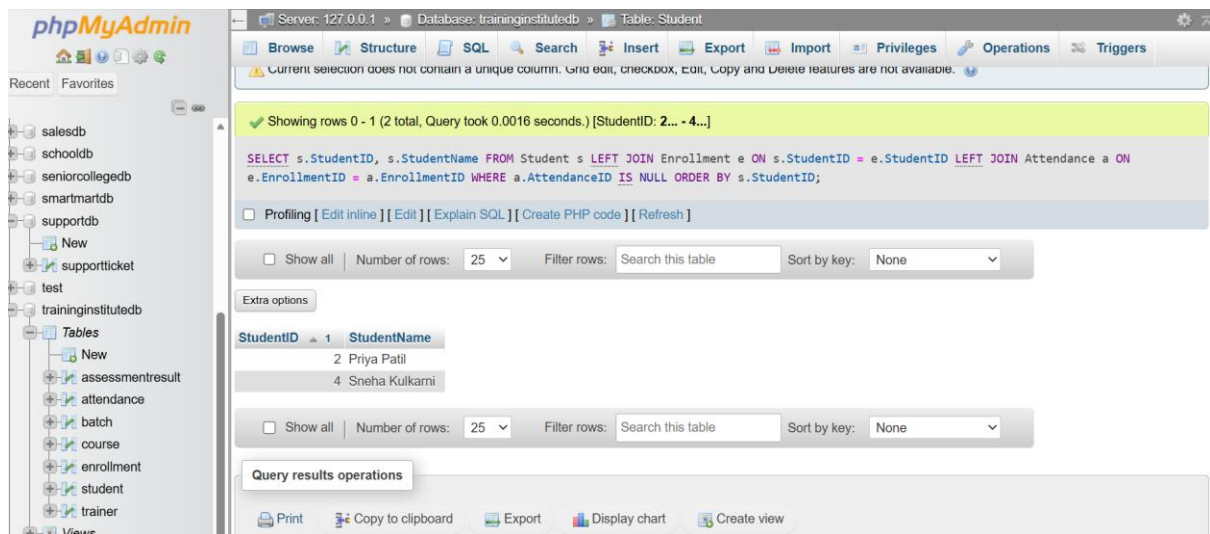
ON s.StudentID = e.StudentID

LEFT JOIN Attendance a

ON e.EnrollmentID = a.EnrollmentID

WHERE a.AttendanceID IS NULL

ORDER BY s.StudentID;



Course-wise Performance Analysis

QUERY

SELECT

c.CourseID,

c.CourseName,

ROUND(AVG(ar.Marks), 2) AS AverageScore,

COUNT(DISTINCT e.StudentID) AS TotalStudents

FROM Course c

JOIN Batch b

ON c.CourseID = b.CourseID

JOIN Enrollment e

ON b.BatchID = e.BatchID

JOIN AssessmentResult ar

ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

c.CourseID,

c.CourseName

ORDER BY AverageScore DESC;

The screenshot shows the phpMyAdmin interface with the 'Course' table selected in the 'traininginstitutedb' database. The SQL query displayed is:

```
SELECT c.CourseID, c.CourseName, ROUND(AVG(ar.Marks), 2) AS AverageScore, COUNT(DISTINCT e.StudentID) AS TotalStudents FROM Course c JOIN Batch b ON c.CourseID = b.CourseID JOIN Enrollment e ON b.BatchID = e.BatchID JOIN AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID GROUP BY c.CourseID, c.CourseName ORDER BY AverageScore DESC;
```

The results table shows the following data:

CourseID	CourseName	AverageScore	TotalStudents
2	Web Development	86.83	3
4	Database Management	82.00	2
3	Cybersecurity	78.83	3
1	Python Data Science	73.64	7

Overall Student Ranking

QUERY

SELECT

s.StudentID,

s.StudentName,

ROUND(AVG(ar.Marks), 2) AS AverageScore,

RANK() OVER (

ORDER BY AVG(ar.Marks) DESC

) AS StudentRank

FROM Student s

JOIN Enrollment e

ON s.StudentID = e.StudentID

JOIN AssessmentResult ar

ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

s.StudentID,

s.StudentName

ORDER BY StudentRank;

Server: 127.0.0.1 » Database: traininginstitutedb » Table: Student

Showing rows 0 - 9 (10 total, Query took 0.0022 seconds.)

```

SELECT s.StudentID, s.StudentName, ROUND(AVG(ar.Marks), 2) AS AverageScore, RANK() OVER ( ORDER BY AVG(ar.Marks) DESC ) AS StudentRank
FROM Student s JOIN Enrollment e ON s.StudentID = e.StudentID JOIN AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID GROUP BY
s.StudentID, s.StudentName ORDER BY StudentRank;

```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

StudentID	StudentName	AverageScore	StudentRank
10	Pooja Gupta	93.00	1
1	Rahul Sharma	92.00	2
3	Amit Joshi	88.50	3
2	Priya Patil	83.25	4
5	Rohan Deshmukh	81.75	5
9	Aditya Pawar	78.00	6
8	Neha Jadhav	76.00	7
6	Anjali More	72.00	8
4	Sneha Kulkarni	65.75	9
7	Kunal Shah	47.50	10

Average Attendance by Batch

QUERY

SELECT

b.BatchID,

b.BatchName,

ROUND(

AVG(

CASE

WHEN a.Status = 'Present' THEN 100

ELSE 0

END

),

2

) AS AverageAttendance

FROM Batch b

JOIN Enrollment e

ON b.BatchID = e.BatchID

JOIN Attendance a

ON e.EnrollmentID = a.EnrollmentID

GROUP BY

b.BatchID,

b.BatchName

ORDER BY AverageAttendance DESC;

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Table: Batch' is selected. The SQL query is displayed in the query editor, and the results are shown in a table below. The query calculates the average attendance for each batch, rounded to 2 decimal places. The results table has columns: BatchID, BatchName, and AverageAttendance. The data is sorted by AverageAttendance in descending order.

BatchID	BatchName	AverageAttendance
5	Cybersecurity Batch	66.67
1	PDS Morning Batch	62.50
2	PDS Evening Batch	58.33
3	Web Development Batch	50.00

Student-wise Training Progress

QUERY

SELECT

s.StudentID,

s.StudentName,

b.BatchName,

c.CourseName,

ROUND(AVG(ar.Marks), 2) AS AverageScore

FROM Student s

JOIN Enrollment e

ON s.StudentID = e.StudentID

JOIN Batch b

ON e.BatchID = b.BatchID

JOIN Course c

ON b.CourseID = c.CourseID

LEFT JOIN AssessmentResult ar

ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

s.StudentID,

s.StudentName,

b.BatchID,

b.BatchName,

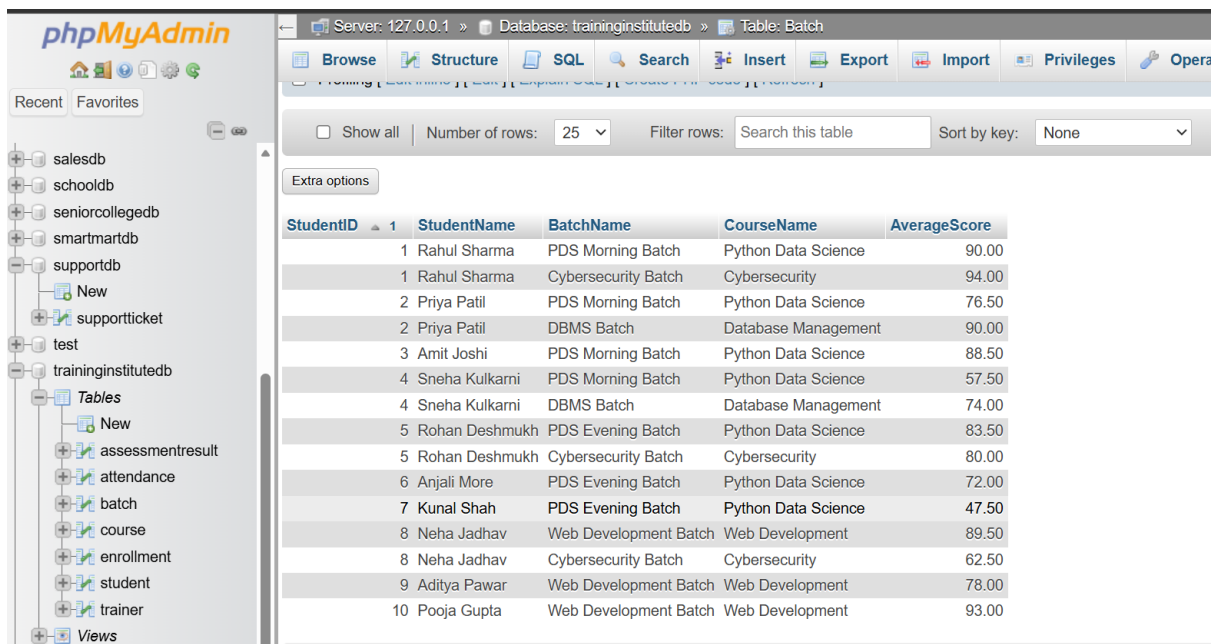
c.CourseID,

c.CourseName

ORDER BY

s.StudentID,

b.BatchID;



Server: 127.0.0.1 » Database: traininginstitutedb » Table: Batch

Extra options

StudentID	StudentName	BatchName	CourseName	AverageScore
1	Rahul Sharma	PDS Morning Batch	Python Data Science	90.00
1	Rahul Sharma	Cybersecurity Batch	Cybersecurity	94.00
2	Priya Patil	PDS Morning Batch	Python Data Science	76.50
2	Priya Patil	DBMS Batch	Database Management	90.00
3	Amit Joshi	PDS Morning Batch	Python Data Science	88.50
4	Sneha Kulkarni	PDS Morning Batch	Python Data Science	57.50
4	Sneha Kulkarni	DBMS Batch	Database Management	74.00
5	Rohan Deshmukh	PDS Evening Batch	Python Data Science	83.50
5	Rohan Deshmukh	Cybersecurity Batch	Cybersecurity	80.00
6	Anjali More	PDS Evening Batch	Python Data Science	72.00
7	Kunal Shah	PDS Evening Batch	Python Data Science	47.50
8	Neha Jadhav	Web Development Batch	Web Development	89.50
8	Neha Jadhav	Cybersecurity Batch	Cybersecurity	62.50
9	Aditya Pawar	Web Development Batch	Web Development	78.00
10	Pooja Gupta	Web Development Batch	Web Development	93.00

Students Performing Above Overall Average

QUERY

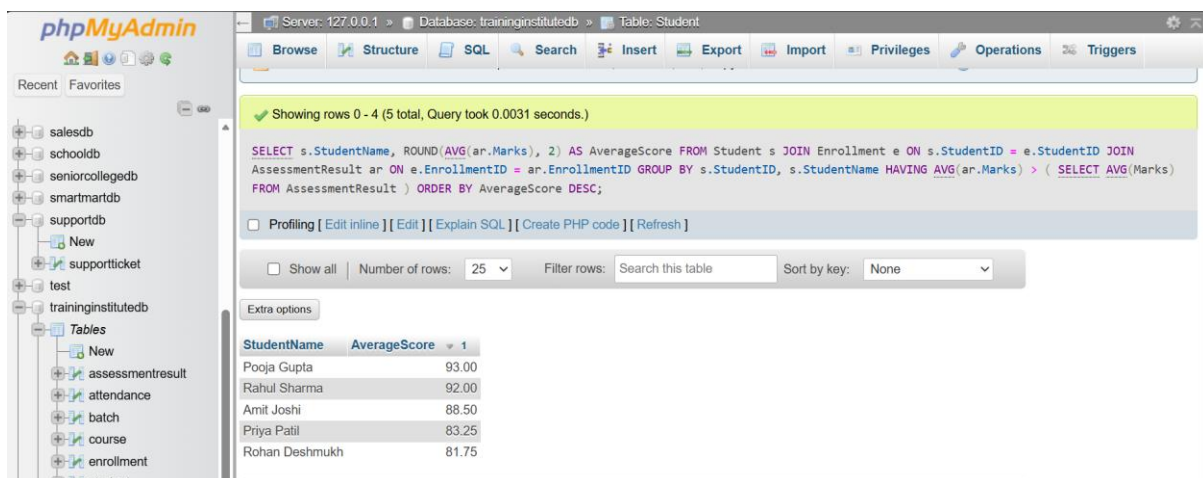
SELECT

s.StudentName,

```

ROUND(AVG(ar.Marks), 2) AS AverageScore
FROM Student s
JOIN Enrollment e
    ON s.StudentID = e.StudentID
JOIN AssessmentResult ar
    ON e.EnrollmentID = ar.EnrollmentID
GROUP BY
    s.StudentID,
    s.StudentName
HAVING AVG(ar.Marks) > (
    SELECT AVG(Marks)
    FROM AssessmentResult
)
ORDER BY AverageScore DESC;

```



Showing rows 0 - 4 (5 total, Query took 0.0031 seconds.)

```

SELECT s.StudentName, ROUND(AVG(ar.Marks), 2) AS AverageScore FROM Student s JOIN Enrollment e ON s.StudentID = e.StudentID JOIN
AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID GROUP BY s.StudentID, s.StudentName HAVING AVG(ar.Marks) > ( SELECT AVG(Marks)
FROM AssessmentResult ) ORDER BY AverageScore DESC;

```

Extra options

StudentName	AverageScore
Pooja Gupta	93.00
Rahul Sharma	92.00
Amit Joshi	88.50
Priya Patil	83.25
Rohan Deshmukh	81.75

Highest Marks Obtained in Each Course

QUERY

```

SELECT
    c.CourseName,
    MAX(ar.Marks) AS HighestMarks
FROM Course c

```

JOIN Batch b

ON c.CourseID = b.CourseID

JOIN Enrollment e

ON b.BatchID = e.BatchID

JOIN AssessmentResult ar

ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

c.CourseID,

c.CourseName

ORDER BY HighestMarks DESC;

The screenshot shows the phpMyAdmin interface. The left sidebar displays a database structure with 'traininginstitutedb' selected. The main panel shows a SQL query that has been executed, resulting in a table of course names and their highest marks. The query is: `SELECT c.CourseName, MAX(ar.Marks) AS HighestMarks FROM Course c JOIN Batch b ON c.CourseID = b.CourseID JOIN Enrollment e ON b.BatchID = e.BatchID JOIN AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID GROUP BY c.CourseID, c.CourseName ORDER BY HighestMarks DESC;` The results table shows four courses: Cybersecurity (95.00), Web Development (94.00), Python Data Science (92.00), and Database Management (91.00).

CourseName	HighestMarks
Cybersecurity	95.00
Web Development	94.00
Python Data Science	92.00
Database Management	91.00

Trainers Handling Multiple Batches

QUERY

SELECT

t.TrainerID,

t.TrainerName,

t.Specialization,

COUNT(b.BatchID) AS TotalBatches

FROM Trainer t

JOIN Batch b

ON t.TrainerID = b.TrainerID

GROUP BY

t.TrainerID,

t.TrainerName,

t.Specialization

HAVING COUNT(b.BatchID) > 1

ORDER BY TotalBatches DESC;

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Table: Trainer' is selected. The SQL query is displayed in the main area:

```
SELECT t.TrainerID, t.TrainerName, t.Specialization, COUNT(b.BatchID) AS TotalBatches FROM Trainer t JOIN Batch b ON t.TrainerID = b.TrainerID GROUP BY t.TrainerID, t.TrainerName, t.Specialization HAVING COUNT(b.BatchID) > 1 ORDER BY TotalBatches DESC;
```

The query results are shown in a table with 4 columns: TrainerID, TrainerName, Specialization, and TotalBatches. There are 2 rows of data.

TrainerID	TrainerName	Specialization	TotalBatches
1	Amit Patil	Python and Data Science	2
2	Snehal Joshi	Web Development	2

Students Enrolled in More Than One Batch

QUERY

SELECT

s.StudentID,

s.StudentName,

COUNT(e.BatchID) AS TotalBatches

FROM Student s

JOIN Enrollment e

ON s.StudentID = e.StudentID

GROUP BY

s.StudentID,

s.StudentName

HAVING COUNT(e.BatchID) > 1

ORDER BY TotalBatches DESC;

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Table: Student' is selected. The SQL query displayed is:

```
SELECT s.StudentID, s.StudentName, COUNT(e.BatchID) AS TotalBatches FROM Student s JOIN Enrollment e ON s.StudentID = e.StudentID GROUP BY s.StudentID, s.StudentName HAVING COUNT(e.BatchID) > 1 ORDER BY TotalBatches DESC;
```

The results show 5 rows, sorted by TotalBatches in descending order:

StudentID	StudentName	TotalBatches
4	Sneha Kulkarni	2
1	Rahul Sharma	2
8	Neha Jadhav	2
5	Rohan Deshmukh	2
2	Priya Patil	2

Create StudentProgress View

QUERY

CREATE OR REPLACE VIEW StudentProgress AS

SELECT

s.StudentID,

s.StudentName,

b.BatchName,

c.CourseName,

ROUND(AVG(ar.Marks), 2) AS AverageScore

FROM Student s

JOIN Enrollment e

ON s.StudentID = e.StudentID

JOIN Batch b

ON e.BatchID = b.BatchID

JOIN Course c

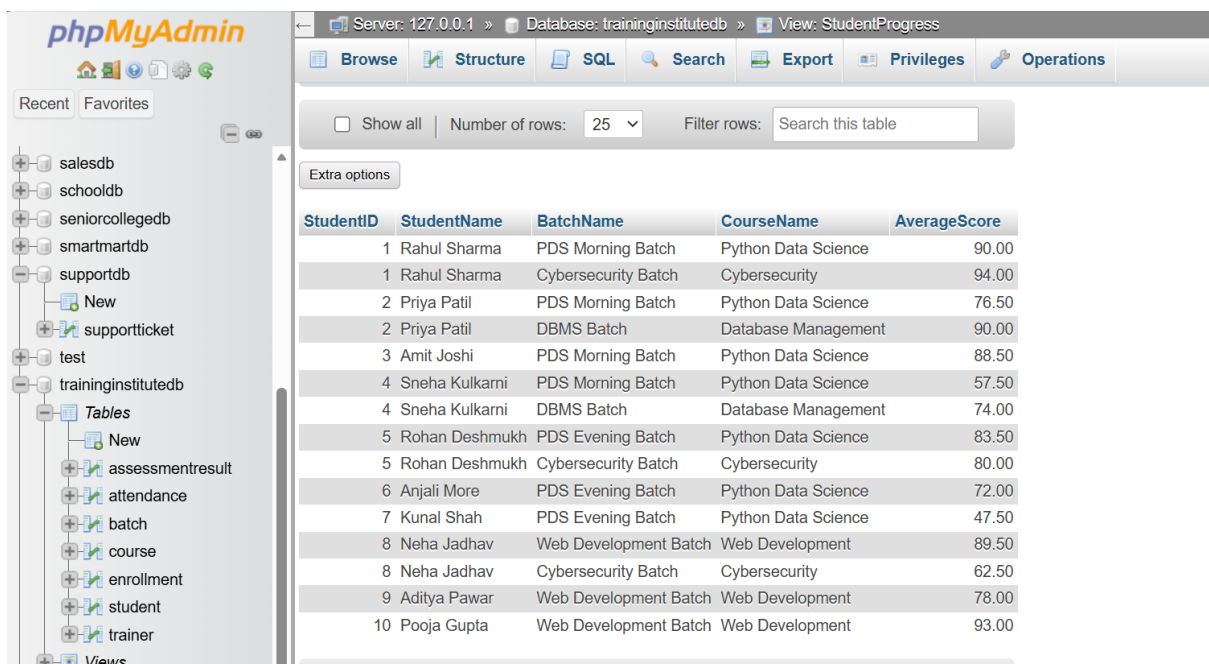
ON b.CourseID = c.CourseID

LEFT JOIN AssessmentResult ar

ON e.EnrollmentID = ar.EnrollmentID

GROUP BY

```
s.StudentID,  
s.StudentName,  
b.BatchID,  
b.BatchName,  
c.CourseID,  
c.CourseName;
```



The screenshot shows the phpMyAdmin interface with the 'traininginstitutedb' database selected. The 'StudentProgress' table is viewed, displaying a list of students and their scores. The table has 5 columns: StudentID, StudentName, BatchName, CourseName, and AverageScore. The data is sorted by StudentID in ascending order.

StudentID	StudentName	BatchName	CourseName	AverageScore
1	Rahul Sharma	PDS Morning Batch	Python Data Science	90.00
1	Rahul Sharma	Cybersecurity Batch	Cybersecurity	94.00
2	Priya Patil	PDS Morning Batch	Python Data Science	76.50
2	Priya Patil	DBMS Batch	Database Management	90.00
3	Amit Joshi	PDS Morning Batch	Python Data Science	88.50
4	Sneha Kulkarni	PDS Morning Batch	Python Data Science	57.50
4	Sneha Kulkarni	DBMS Batch	Database Management	74.00
5	Rohan Deshmukh	PDS Evening Batch	Python Data Science	83.50
5	Rohan Deshmukh	Cybersecurity Batch	Cybersecurity	80.00
6	Anjali More	PDS Evening Batch	Python Data Science	72.00
7	Kunal Shah	PDS Evening Batch	Python Data Science	47.50
8	Neha Jadhav	Web Development Batch	Web Development	89.50
8	Neha Jadhav	Cybersecurity Batch	Cybersecurity	62.50
9	Aditya Pawar	Web Development Batch	Web Development	78.00
10	Pooja Gupta	Web Development Batch	Web Development	93.00